

درس سیستم عامل

تمرین عملی ششم سوال 2

402105727

متین باقری

402105802

فاطمیما تیمارچی

فایل های تغییر داده شده (run.c) و عکس خروجی تمامی قسمت ها را ضمیمه کردم .

(آ)

ابتدا به تگ t4 رفتم :

```
Timarchi-Bagheri@debian:~/zocker$ git pull origin master && git checkout t4
From https://github.com/mghgm/zocker
* branch      master      -> FETCH_HEAD
Already up to date.
Note: switching to 't4'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 0f4e56b Add Task 4
```

(ب)

ایمیج را دریافت کردم :

```
Timarchi-Bagheri@debian:~/zocker$ sudo docker pull alpine:3.22.2
3.22.2: Pulling from library/alpine
Digest: sha256:4b7ce07002c69e8f3d704a9c5d6fd3053be500b7f1c69fc0d80990c2ad8dd412
Status: Image is up to date for alpine:3.22.2
docker.io/library/alpine:3.22.2
```

(ج)

دستور را اجرا کردم :

```

docker: to/ctb/ary/atp/the.3.22.2
Timarchi-Bagheri@debian:~/zocker$ sudo docker run --rm -d alpine:3.22.2 sleep 1000
adad8e44eea31e5a7f103760d862ab7eeb9e7f97907bd2a8a4e05a2617468210
Timarchi-Bagheri@debian:~/zocker$ sudo docker export adad8e44eea3 -o export.tar
Timarchi-Bagheri@debian:~/zocker$ sudo su
root@debian:/home/fatima-t/zocker# docker ps

```

وقتی این دستور را میزنیم عددی که در ترمینال نوشته میشه همون container id هست و این id کامل هست .

سوال :

وقتی که از دستور r استفاده میکنیم اون وقت داکر بعد از اینکه پردازش اصلی کانینر متوقف شد کانینر رو به صورت کامل حذف میکنه . و همین باعث میشه که کانینر دیگه تو خروجی docker ps -a دیده نمیشه و کلا متادیتا و لایه کانینر پاک میشه و دیگه لازم نیست کانینر رو دستی حذف کنیم . در این دستور بخصوص وقتی sleep 1000 تموم بشه یا کانینر stop بشه اون وقت کانینر به صورت خودکار حذف میشه . در کل از فواید rm میتونیم به این اشاره کنیم که باعث میشه سیستم ما شلوغ نشه و اگه ما کانینری برای تست ایجاد میکنیم دیگه لازم نیست در انتهای کار باهانش خودمون دستی اون رو حذف کنیم و خودش به صورت خودکار حذف میشه .

د و ه

از 12 رقم ابتدای خروجی قسمت قبل استفاده کردم و این دستور را اجرا کردم :

```

Timarchi-Bagheri@debian:~/zocker$ sudo docker export adad8e44eea3 -o export.tar
Timarchi-Bagheri@debian:~/zocker$ sudo su
root@debian:/home/fatima-t/zocker# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS   NAMES
adad8e44eea3   alpine:3.22.2 "sleep 1000"            About a minute Up About a minute        heuristic_blackwell
root@debian:/home/fatima-t/zocker# docker exec -it heuristic_blackwell sh
/ # ls /proc
1          cgroups      dma           interrupts    keys          meminfo       partitions   swaps         uptime
13         cmdline     driver        iomem        kmsg          misc          pressure     sys           version
7          consoles   dynamic_debug ioports      kpagecount   modules       schedstat    sysrq-trigger vmallocinfo
acpi       cpuinfo    execdomains  irq          kpagecount   mounts        self         sysvipc       vmstat
asound     crypto     fb           kallsyms     kpageflags   mtrr         slabinfo    thread-self   zoneinfo
buddyinfo devices    filesystems  kcore        loadavg      net           softirqs    timer_list
bus        diskstats  fs           key-users    loadavg      net           softirqs    timer_list
/ #

```

سوال :

مطابق عکس بالا من محتوای پوشه proc یک کانینر در حال اجرا رو در ترمینال نشون دادم و همون طور که میبینید پر از فایل های سیستمی هست . سپس با استفاده از بخش ه اومدم محتوای فایل export.tar رو در یک پوشه خالی به نام tmp/zocker/mycontainer استخراج کردم و با استفاده از دستور ls مطابق شکل زیر دیدم که محتوای پوشه proc خالی هست :

```
root@debian:/home/fatima-t/zocker# mkdir -p /tmp/zocker/mycontainer
root@debian:/home/fatima-t/zocker# tar -xf export.tar -C /tmp/zocker/mycontainer
root@debian:/home/fatima-t/zocker# ls /tmp/zocker/mycontainer/proc
root@debian:/home/fatima-t/zocker#
```

پس در کل نتیجه میگیریم که در کانتینر در حال اجرا `proc = /procfs mount` به عبارت دیگه در اصل یک فایل سیستم مجازیه که توسط `kernel` در زمان اجرا `mount` میشه. پوشه `proc` پر از اطلاعات زنده به صورت فایل و پوشه هست. اما در خروجی دستور `sudo docker export...` پوشه `proc` کاملاً خالیه. چون این دستور فقط محتوای فایل سیستم روی دیسک رو استخراج میکنه و همین باعث میشه که فایل سیستم های مجازی و `\proc` و `\sys` ذخیره نمیشن. به عبارت دیگه `proc` اصلاً فایل های واقعی روی دیسک نیست و در زمان اجرای کانتینر توسط `kernel`, `mount` میشه. بنابراین وقتی دستور `sudo docker export...` رو میزنیم چون `mount` های `runtime` ذخیره نمیشن پس در کل چیزی ذخیره نمیشه و فایل کلاً خالیه.

(و)

برای این قسمت این بخش را به فایل `run.c` اضافه کردم:

```
if (cont.base_dir[0] != '\0') {
    snprintf(container_dir, sizeof(container_dir), "%s", cont.base_dir);
} else {
    if (setup_container_dir(cont.id, container_dir) != 0) {
        fprintf(stderr, "[ERR] Failed to setup container directory for %s\n", cont.id);
        return 1;
    }
}
```

منطق کل کنم به این صورت هست که اگر در دستور `base-dir` نبود که هیچی آگه بود چون کلاً این پرچم به این معنی هست که مسیری که کانتینر استفاده میکنه باید از مسیر سیستم میزبان برداشته باشه برای همین در کم ابتدا با استفاده از پرچم `base-dir` اومدم به جای اینکه برنامه بره داخل مسیر پیش فرض کارش رو پیش ببره بره مسیر آماده `rootfs` رو برداره و آگه هم `rootfs` وجود نداشت خودش با استفاده از `setup_container_id` یکی بسازه و از اون استفاده کنه. (کل فایل `run.c` رو ضمیمه کردم)

پس از تغییر کد خروجی به صورت زیر میشه:

```
Timarchi-Bagheri@debian:~/zocker$ make
gcc -Wall -Wextra -std=c99 -MMO -MP -c src/run.c -o src/run.o
gcc -Wall -Wextra -std=c99 -MMO -MP -o zocker src/config.o src/main.o src/run.o src/setup.o
sudo setcap cap_sys_admin,cap_sys_chroot+ep zocker
```

```
Timarchi-Bagheri@debian:~/zocker$ make
make: 'zocker' is up to date.
Timarchi-Bagheri@debian:~/zocker$ ./zocker run --name my-container --base-dir "/tmp/zocker/mycontainer" 'sh'
Running container 'my-container' with PID: 1
Timarchi-Bagheri@debian:/$ exit
[Parent] Container my-container stopped
Timarchi-Bagheri@debian:~/zocker$ ./zocker run --name my-container --base-dir "/tmp/zocker/mycontainer" 'sh'
```

در اینجا من ابتدا دستور `make` رو زدم و سپس دستور مورد نظر رو وارد کردم و همون طور که در عکس بالا مشاهده میکنید کانتینر اجرا شد و در نهایت هم با زدن دستور `exit` اجرای کانتینر را متوقف کردم

سوال :

بله . اما یه شرط داره که این شرط همونی هست که در فایل run.c تغییر دادیم . این شرط این هست zocker باید طوری کانتینر رو اجرا کنه که rootfs همون دایرکتوری پایه روی میزبان باشه . به عبارت دیگه با زدن دستور — base-dir ما یک مسیر مشخص میکنیم که این مسیر توسط chroot به دست میاد . با اینکار اون دایرکتوری به عنوان root فایل سیستم کانتینر ما انتخاب میشه و هر فایلی که روی میزبان داخل این دایرکتوری یا زیرشاخه هاش ایجاد میشه از داخل کانتینر قابل مشاهده هست و میتونیم بهش دسترسی داشته باشیم و هر فایلی که خارج از این مسیر باشه بعد از chroot دیگه قابل مشاهده نیست به عبارت دیگه بعد از chroot کانتینر دیگه هیچ دیدی به فایل های خارج از این دایرکتوری نداره .

حالا بخش test.txt :

برای این بخش زمانی که کانتینر در حال اجرا بود فایل test.txt رو به صورت مستقیم ساختم . بعد برای فهمیدن اینکه به فایل دسترسی دارم یا نه دستور ls -l /test.txt رو اجرا کردم و سپس محتویاتش رو با دستور cat /test.txt چک کردم و خروجی به صورت زیر شد :

```
Timarchi-Bagheri@debian:~/zocker$ ./zocker run --name my-container --base-dir "/tmp/zocker/mycontainer" 'sh'
Running container 'my-container' with PID: 1
Timarchi-Bagheri@debian:/$ ls
bin      etc      lib      mnt      proc     run      srv      test.txt  usr
dev      home    media    opt      root     sbin     sys      tmp       var
Timarchi-Bagheri@debian:/$ ls -l /test.txt
-rw-r--r-- 1 root root 6 Dec 15 08:40 /test.txt
Timarchi-Bagheri@debian:/$ cat /test.txt
hello
Timarchi-Bagheri@debian:/$
```

همون طور که در عکس بالا مشاهده میکنید من از داخل کانتینر به فایلی که مستقیم به مسیر اضافه کردم دسترسی دارم .

(ز)

سوال :

خیر . داکر واقعی متکی بر chroot نیست و از mount namespace و user namespace و overlayfs و ... استفاده میکنه و chroot بر ازش فقط یک ابزار کمکی هست . لایه های ایمپج مشترک است و read-only است و لایه کانتینر تنها یک لایه writable و اختصاصی دارد و تنها در موارد خاص Copy-on-write داریم . پس داکر واقعی isolation داره و سر همین سرعش هم بیشتره

دلیل ناکارآمدی : مشکل این روش دقیقا تو زمان اجرا شدن دو کانتینر با ایمپج یکسان رخ میده . اگه دو کانتینر با ایمپج یکسان اجرا بشن دو حالت ممکنه پیش بیاد :

1- آگه هر دوی کانتینر ها از یک base-dir مشترک استفاده کنند اون وقت فایل سیستم آن ها مشترک هست و همین باعث میشه تغییرات یک کانتینر روی دیگری اثر بزاره و همین باعث میشه دیگه ما isolation نداشته باشیم

2- حالا حتی آگه بخوام مشکل حالت 1 رو رفع کنم به این صورت که برای اینکه داده های دو کانتینر روی هم اثر نزارن پیام برای هر کانتینر یک کپی جدا از rootfs ایمج بسازم که اینکار باعث مصرف زیاد دیسک میشه همچنین به دلیل کپی کردن roofs ایمج زمان راه اندازی بیشتر میشه و در نتیجه مدیریت هم سخت تر میشه
